# R47/C47 RPN solves, integration, plotting

This appnote covers solves, integration and plotting ***using RPN programmed functions***. This is ***different*** from the same functionality ***using the EQN menu***, which use algebraic methods. We show manual and programmed usage of MVAR-based RPN programs. We show:

- RPN user function/program to manually use SOLVE from the UI,
- RPN user function/program to manually use ∫f d from the UI,
- Automate or sequence the above in RPN programs,
- Graph an RPN function/program, by minimal changes to an existing demo program:
    - change 4 steps 0001, 0002, 0032 & 0034
    - delete existing steps 0036 – 0053
    - add 3 steps from 0036

## Table of Contents

R47/C47 RPN solves, integration, plotting ..... 1
Manually solve and integrate on R47/C47 ..... 2
Start with your RPN equation ..... 2
RPN function/program structure required ..... 2
Solve for the roots ..... 2
Get the surface area ..... 3
Programming the roots and integrals ..... 3
Run your program ..... 4
Graphing RPN programs ..... 4
Graphing using STAT and PLOT menus ..... 4
Simple example for the graphing using PLSTAT ..... 4
Graphing the prior RPN function 'FUNC' using the STAT and PLOT menus ..... 5
Plot it ..... 6

# Manually solve and integrate on R47/C47

## Start with your RPN equation

$f(x) = x^2 + 2x - 2 = 0$

Make an RPN program to calculate the expression f(x), which will be solved as f(x) = 0. Only a single root per solve will be found and you will need to guide the solver where to look next after finding one root.

## RPN function/program required

The program is required to declare all variables using MVAR. Use RCL after that to use the variables. This method will include all variables you may want to appear in the solve menu:

```
C47 Program file export: Export format version 3, C47 program version 1.
0000:  Prgm #2/2: 34 bytes / 13 steps

0001:  LBL 'FUNC'
0002:    MVAR 'x'
0003:    RCL 'x'
0004:    ENTER
0005:    x²
0006:    x⇄y
0007:    2
0008:    ×
0009:    +
0010:    2
0011:    -
0012:  END
```

**Entering the program:**

From run mode, press XEQ .. to open a new program space, then f PRGM to enter program mode. After entering the program, press f PRGM again to return to run mode.

*A typical program for the above quadratic expression*

- The MVAR 'x' command is needed to tell the R47/C47 to stop and display a menu when SOLVE or ∫f d is used.
- All declared MVAR variables will show in a menu, only 'x' declared in this case.
- After MVAR you must use RCL 'x' to be able to use the x-value as usual on the stack.

## Solve for the roots

- Pick starting points for the solver, of x = -2 to 2 to kick off the search for roots:

  ADV SOLVE α FUNC ENTER 2 CHS [x] 2 [x] [x] STO 00

  Root r1: x = 0.7320508075688772935274463415058724 stored to R00

- Guide the solver to a different part of the function to search for a second root, eg. pick more negative points to seek the other root:

  10 CHS [x] 11 CHS [x] [x] STO 01

  Root r2: x = -2.7320508075688772935274463415058 72 stored to R01

## Get the surface area

- Get the surface area between the curve and the x-axis, between the two roots:

  ∫f d α FUNC ENTER RCL 00 [x] RCL 01 [x] [x] ∫x y STO 02

  Area: ∫x dx = 6.928203230275509174109785366023489 stored to R02

# Programming the roots and integrals

Create a program to automate the steps above to find the roots, store them (as above) to R00, R01 & R02, and find the area between the roots.

```
C47 Program file export: Export format version 3, C47 program version 1.
0000:  Prgm #1/1: 146 bytes / 35 steps

0001:  LBL 'INTDIF'
0002:    PGMSLV 'FUNC'
0003:    -2
0004:    2
0005:    SOLVE 'x'
0006:    STO 00              // store r1
0007:    -11
0008:    -10
0009:    SOLVE 'x'           // store r2
0010:    STO 01

0011:    PGMINT 'FUNC'
0012:    RCL 00              // lower limit being r1 in Y
0013:    RCL 01              // upper limit being r2 in X
0014:    ∫x y 'x'
0015:    STO 02              // store integral
0016:    'x²+2x-2:↵∫=↵r2=↵r1='
0017:    STO 04              // store 4 lines of user display text, separated by ↵: T↵X↵Y↵Z
0018:    CLSTK
0019:    RCL 00              // show root 1
0020:    RCL 01              // show root 2
0021:    RCL 02              // show integral
0022:    AVIEW 04            // show user text
0023:  RTN

0024:  LBL 'FUNC'
0025:    MVAR 'x'
0026:    RCL 'x'
0027:    ENTER
0028:    x²
0029:    x⇄y
0030:    2
0031:    ×
0032:    +
0033:    2
0034:    -
0035:  .END.
```

**Pro tip 1:**

Create the second program within the programming space of the first. This keeps related functions together in one file when saving (WRITEP) or exporting (XPORTP).

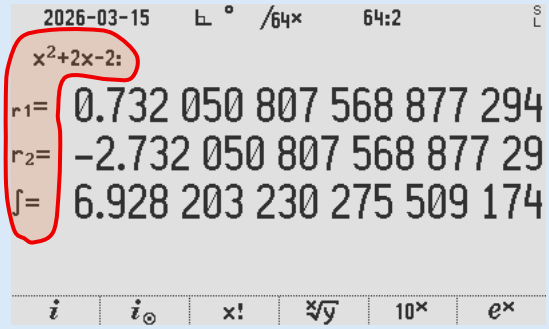
**Pro tip 2:**

For quick access to commands, functions, and variables, type **α** followed by the name directly. For example after XEQ, SOLVE and ∫f d (marked in red). This is faster than navigating menus, though menu selection works equally well.

*Typical root find and integral RPN program: Both* INTDIF *and* FUNC *in the same program space.*

## Run your program

Get the result by typing: `XEQ` `α` `INTDIF` `ENTER`




`INTDIF` *automated root find and output*

**Pro tip 3:**

Create custom user labels in step 0016 and label the stack elements in the sequence of TXYZ in step 0022 above. The mapping is shown in colours as follows:

$x^2+2x-2$:↵∫=↵$r_2$=↵$r_1$=

# Graphing RPN programs

R47/C47 has no plotting function specifically for user RPN programs. The EQN menu plots algebraic formulas, not RPN program output. To plot an RPN function, the EQN menu cannot be used and a manual plotting loop is the only way to go.

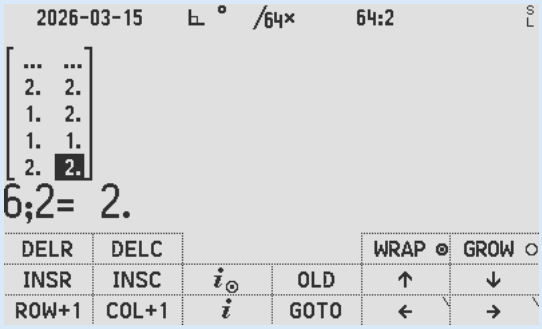
## Graphing using STAT and PLOT menus

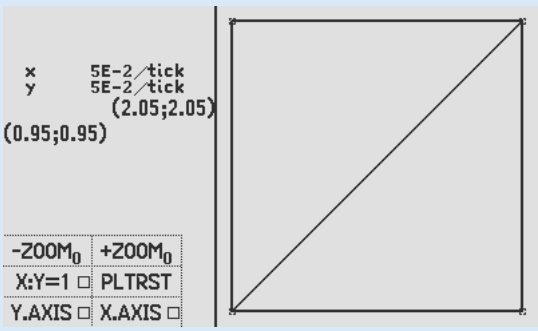
The easiest method is to use the STAT functions including PLSTAT to draw a graph, essentially by entering some (x,y) pairs into the STAT matrix and plotting.

## Simple example for the graphing using PLSTAT

As a manual illustration of plotting from the STATS matrix, let us create a matrix of 6 rows by 2 columns and populate it manually with a graph resembling a square:

`CLΣ` `MATX` `6` `ENTER` `2` `NEW` `EDIT` `1` → `1` → `2` → `1` → `2` → `2` → `1` → `2` `1` → `1` `2` → `2`
`EXIT` `STO` `α` `'STATS'` `PLOT` `PLSTAT`




*Edit a new matrix, exit the editor, store to* `STATS`*, and plot using* `PLSTAT`

*Example of an arbitrary plot using PLSTAT*

## Graphing the prior RPN function '**FUNC**' using the STAT and PLOT menus

As an easy start, this appnote guides you to load up the **BinetV3.p47** program (in the PROGRAMS folder in the distro zip file) and minimally modify that program to plot your '**FUNC**' program: Changes to the Binet program are in yellow, with removals and changes as indicated. Grey text below is unchanged from referenced program BinetV3.p47:

```
C47 Program file export: Export format version 3, C47 program version 1.
0000:  Prgm #43: 171 bytes / 55 steps

0001:  LBL 'Binet'            // Change to 'PLOTABC' or whatever you like
0002:    REM 'Plot the Binet function, V03.
0003:    LocR 11              // Define sufficient local registers
0004:    STOCFG .02
0005:    STOS .03             // Store you stack and config before wrecking it
0006:    CLΣ                  // Clear the STAT memory
0007:    101                  // Define 100 steps from 1 to 101
0008:    STO .00              // Loop counter start in .00 (i = 101)
0009:    STO .01              // Loop origin in .01 (101)

0010:  LBL B                  // Graph loop start
0011:    RCL .01
0012:    RCL .00
0013:    -                    // Convert the loop counter to X (101-i), start=0 end=100
0014:    RCL .01              // Load the original max counter value to X
0015:  XEQ A                  // Run the RPN math function, with Y=0-100 and X=101
0016:    AVIEW .00            // Show the status top left
0017:    Σ+                   // Load the pair as new STAT point
0018:    DSZ .00              // Subtract from loop counter i, break when 0 reached
0019:      GTO B

0020:    RCLS .03             // Restore your stack and config to pre-loop
0021:    RCLCFG .02
0022:    'PRESS R/S TO PLOT'
0023:    STO 00
0024:    DROPx
0025:    PROMPT 00            // Wait for user to press R/S
0026:    PLSTAT               // Display graph
0027:  RTN

0028:  LBL A                  // This is the part which you customize to suit
0029:    REM 'CALC P1'        // Your range r1 to r2 crossing the x-axis, say -3 to 3
                              // Incoming values: Y = 0-100 and X = 101
                              //  by ( Y/(X-1)*6-3 ) convert to a range from -3 to 3
0030:    DECR X
0031:    ÷
0032:    7                    // Change: to 6, being (3-(-3))
0033:    ×
0034:    2                    // Change: to 3, being the negative graph offset
0035:    -

0036:    ENTER↑               STO 'x'        // Add: store the new x value
```

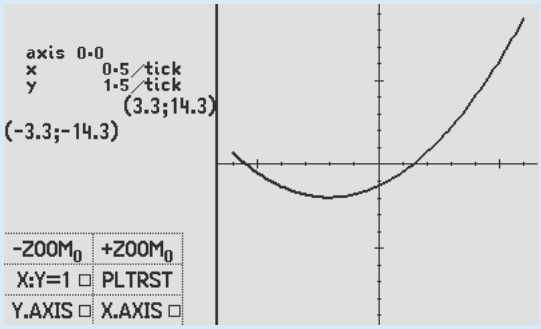
```
0037:   # 74 φ r.golden     XEQ 'FUNC'    // Call your RPN math function
0038:   x≷y                 RCL 'x'       // Push the x-coordinate to X
...                         remove all steps in red
0053:   Re
0054:  RTN
0055:  .END.
```

*Modified BinetV3.p47 program*

## Plot it

Get the graphic result by typing:

**XEQ α PLOTABC ENTER** to run the RPN program sequencing the plot below:




*Output plot of the new* **PLOTABC** *RPN program*

**Pro tip 4:**

Try the **PLSTAT** menu option to see what they do. **PLTRST** (Plot Reset) will reset the view settings, so feel free to experiment.

- Once the **STATS** matrix exists, it can be copied to any register for later view. Example: use **RCL α STATS STO 00**

- You can **RCL** a stored matrix, **STO** it as the **STATS** matrix, and view using **PLSTAT**. Example: use **RCL 00 STO α STATS**

- Once a matrix is stored into **STATS**, all accumulators are re-calculated, after which **PLSTAT** is available to use.

Author Jaco Mostert

Copyright (C) 2026 The R47/C47 Development Team. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license can be downloaded from gnu.org.